Nitsche dynamic rupture: profiling and speed-up plan

Where TPV102, TPV104 and TPV205 (p = 1, 2) spend their time at `53ba400`, and what to change first · revision 1, 17 Oct 2026

Revision 1. First version. Plan only: no solver code is changed. One benchmark test (`tests/unit/bench_nitsche_kernels.cpp`) backs the numbers and is not yet committed.

1. Summary

Three costs set the wall time of the benchmark runs.

1. **The two sparse matrix-vector products** Ku and $K_\eta v$ take 84 to 91% of every time step (perf, §4.2). They run at 47 to 66% of the machine's memory bandwidth, so they are memory-bound. About 80% of the matrix entries are face couplings, which MFEM cannot do matrix-free on tets.
2. **Fault output** takes 11 to 26% of the wall time on Vista (all five gold logs, §4.1). This is 0.48 to 1.59 s per frame on rank 0 while every other rank waits.
3. **The time step is set by a few sliver tets.** On the 200 m TPV205 mesh the smallest inscribed size is 26 m against a median of 182 m (§4.6). On the 1000 m mesh, gmsh quality options alone raise the global dt by 1.68×.

The fault itself (traction, slip load, friction) is only 3 to 9% of a serial step, but it all sits on the 20 to 34% of ranks that own fault faces. The fault traction call can be made 36 to 97× faster with a precomputed per-face operator, and the test confirms it matches to about 10^{-15} (§4.3).

#	Change	Expected gain (TPV205 200 m p1, 1102 s)	Changes results?	Effort
1	Cheaper fault output: write cell arrays directly, no deflate, fewer fields; later asynchronous	about 200 to 250 s (18 to 23%), inferred from §4.5	No (output only)	Small
2	Better mesh quality (gmsh optimiser on the half mesh before mirroring)	1.7× larger dt; about 2× with fewer tets (1000 m test)	Yes, new mesh: re-run benchmarks	Small
3	Fault traction with precomputed per-face operators, no unused diagnostics, no whole-vector copy	3 to 5% serial; more on fault ranks	No (exact to round-off)	Medium
4	One fused sweep for $Ku + K_\eta v$	1.10 to 1.26× on the SpMV	No	Medium
5	Free the unused assembly objects and the unused AMG solvers	Part of the 68 to 200 s setup, and about half the memory, so fewer nodes	No	Small
6	Partition weights for fault elements	Up to the 1.1 to 1.6× excess seen at high rank counts	No	Small
7	Local time stepping (clustered, rate 2)	2× with 2 clusters, 4.6× ideal on 200 m	Yes (new scheme)	Large
8	Custom matrix-free or dense-block SIPG kernels (Tandem/SeisSol style)	1.1 to 1.4× measured for dense blocks; more for a full matrix-free kernel	No	Large

The recommended order is 1, 5, 3, 4 and 6 first. These leave the solution unchanged, so the existing golds stay valid, and together they save an estimated 25 to 40% of wall time (inferred: the sum of the separate estimates, not a measured run). Then 2, which needs new benchmark runs. Items 7 and 8 are larger projects for later.

2. What one time step does

The driver calls `ode.Step` once per step (nitsche_dyn_driver.cpp:1444). Central difference calls `NitscheWaveOperator::ImplicitSolve` (nitsche_wave_operator.hpp:273), which advances the fault and then computes the acceleration. The pseudocode below shows the work in order, with the share of a serial p = 1 step from §4.2.

```
step(u, v, t):                                          share of step (p=1)
  # AdvanceFault_  (nitsche_wave_operator.hpp:821-925)
  t_u   = ComputeTractionDiagnostics(u)                 ┐ traction 4.7-5.4%
  t_kv  = ComputeTractionDiagnostics(v)   (KV part)     ┘
  for each fault DOF: friction closure, update slip     closure < 0.5%
  MPI_Allreduce(max |V|, ...)                           (nitsche_wave_operator.hpp:909)
  # Acceleration_  (nitsche_wave_operator.hpp:783-818)
  r  = K u            (HypreParMatrix SpMV)             ┐ SpMV 84-87%
  r += K_eta v        (second SpMV, Kelvin-Voigt)       ┘
  r -= G d + G_eta V  (AssembleSlipOnlyRHS, twice)      slip load 1-4%
  r += Dirichlet RHS  (AssembleDirichletOnlyRHS)        ~0
  r -= C v            (absorbing boundary)              ~0
  a  = M^{-1} r       (NitscheMass::Apply, block diag)  2%
  vector updates u, v                                   4-5%
  # driver, after the step
  station averaging   (every step, collectives)         ~0 serial
  every 0.02 s: fault VTKHDF frame (rank 0)             11-26% of wall on Vista
```

Each of the two traction calls does the following (elasticity_operator_traction.inl:661):

- It builds a whole-space `ParGridFunction`, copies the full vector into it, and calls `ExchangeFaceNbrData` for every shared face, not just fault faces (:1221-1222).
- For each fault face it rebuilds the face transformations, inverts both element Jacobians (dg_elasticity_ip_combined_integrator.hpp:485-489), and assembles and inverts a face mass matrix (:715).
- It also computes decomposed diagnostic tractions that the step does not use.

All of this is the same every step on a fixed mesh, so it can be computed once.

Setup assembles K and K_η lazily on the first step (elasticity_operator_assembly.inl:9-86, ParallelAssemble :216). Both operators are built with `SolverType::CG_AMG` (nitsche_wave_operator.hpp:200, :659), so each one also builds a BoomerAMG hierarchy (:421) that the explicit scheme never uses. Each one also keeps the serial bilinear form and its local matrix (`cached_a_`) after the parallel matrix exists.

The fault writer gathers the vertices and fields to rank 0 every frame (fault_vtu_binary.hpp:396-411). It then writes them through `ParaViewHDFDataCollection::Save`, which evaluates every field cell by cell, applies deflate (level 6 by default, nitsche_dyn_driver.cpp:1348) and flushes the file. Twelve fields are written, two of which (`param_a`, `param_Dc`) never change.

3. How it was measured

- **Vista timings** come from the driver logs of the five gold runs (results/driver.log in each gold folder on `53ba400`). Setup is the wall time at step 1000 minus 1000 times the per-step rate between steps 1000 and 2000.
- **Local profiles:** serial perf (6.8, 499 Hz, call graphs) of the production driver built from `53ba400`. Builds used MFEM 4.9.1, HYPRE 2.28, HDF5 1.14.6 and `-O3 -march=native`. The meshes are coarse enough to fit 15 GB:
  - p = 1: TPV205, TPV104 and TPV102 at 1000 m (187k to 215k tets).

    - p = 2: TPV205 and TPV104 at 3000 m (30k to 33k tets).

  Samples are bucketed by call stack into the rows of §4.2.
- **Kernel benchmark:** `seas_bench_nitsche_kernels` (§6) times each kernel in isolation on the production operators and checks every proposed replacement against them.

Local runs are short, so setup is a larger share than on Vista. §4.2 therefore reports the step loop and setup separately.

4. Results

4.1 Vista gold runs

Run (job)	Ranks	Steps	Wall	Setup	Fault output	Step compute	µs per tet-step per core
TPV205 200 m p1 (1076378)	576	41,614	1102 s	68 s	292 s (26%), 0.48 s/frame	17.8 ms	5.4
TPV205 100 m p1 (1076419)	512	110,004	5276 s	200 s	958 s (18%), 1.59 s/frame	37.4 ms	3.5
TPV104 200 m p1 (1075919)	512	48,961	1641 s	92 s	429 s (26%), 0.72 s/frame	22.9 ms	4.8
TPV205 200 m p2 (1075916)	2304	88,725	2670 s	89 s	293 s (11%), 0.49 s/frame	25.8 ms	31.4
TPV104 200 m p2 (1075917)	2048	104,388	3785 s	120 s	418 s (11%), 0.70 s/frame	31.1 ms	25.8

Fault output is about 600 frames in every run. The per-frame cost grows with the fault size (26k faces at 200 m, 99k at 100 m), not with the order, so it is fixed overhead that a faster solver would not shrink. The per-core rate at p = 1 (3.5 to 5.4 µs per tet-step) matches the local serial rate (4.9 µs). This means the Vista runs at p = 1 scale well. At p = 2 on 2048 to 2304 ranks the rate is 1.2 to 1.6× the local serial rate (20 to 22 µs). Load imbalance and communication are the likely causes (§4.4; inferred).

4.2 Where a step goes (serial perf)

Bucket	TPV205 p1	TPV104 p1	TPV102 p1	TPV205 p2	TPV104 p2
SpMV Ku, $K_\eta v$	**86.8%**	**86.7%**	**83.7%**	**91.2%**	**91.0%**
Fault traction (u and KV)	4.7%	5.1%	5.4%	2.4%	2.9%
Vector updates	4.8%	4.5%	4.4%	3.4%	3.2%
Mass inverse	2.3%	2.0%	2.1%	2.3%	2.0%
Slip load	1.1%	1.0%	3.8%	0.5%	0.5%
Friction closure	0.0%	0.5%	0.4%	0.0%	0.2%
Fault output, stations, Dirichlet RHS	0.2%	0.2%	0.2%	0.2%	0.1%
Setup, share of the whole local run	*36%*	*37%*	*37%*	*62%*	*54%*

Shares are of the step loop, from samples whose call stack is under `NewmarkSolver::Step` or the output calls. The fault buckets shrink on the real meshes, because the fault-face to tet ratio falls as the mesh is refined (1.4% at 200 m). The local runs wrote only a few frames, so output is small here; §4.1 gives its real share. In the 4-rank run, the traction bucket also includes MPI wait in `ExchangeFaceNbrData`. Of the local setup time, 38 to 55% is kernel page faults from allocating the assembly objects.

4.3 Kernel benchmark

Kernel	p = 1 (TPV205 2000 m, 62.6k tets, 392 fault faces)	p = 2 (TPV205 5000 m, 11.2k tets, 82 fault faces)
Matrix size per tet (one matrix)	707 nnz, 8.5 kB	4324 nnz, 52 kB
Ku, share of stream bandwidth	127 ms, 47%	86 ms, 58 to 66%
Fused $Ku + K_\eta v$ in one sweep	1.10 to 1.26× faster than two SpMVs	1.10 to 1.26×
Volume part only: MFEM partial assembly	0.86 to 1.03× (no gain)	0.86 to 1.03×
Volume part only: dense element blocks	1.13 to 1.44×	1.13 to 1.44×
Fault traction, current call	14.3 µs/face (13.2 µs without the vector copy)	24.2 µs/face (22.0 µs)
Fault traction, precomputed per-face operator	**97× faster**, difference ~1e-15	**66× faster**, difference ~1e-15
Traction without the unused diagnostics	1.02 to 1.34× faster	
Slip load (one call)	2.9 ms	1.4 ms

The volume (element) integrals are only about 20% of the nonzeros. The face couplings are about 80% of the nonzeros and about 85% of the SpMV time. This is why MFEM's partial assembly, which covers only the volume part on tets, gains nothing. In MFEM 4.9 and 4.10, `DGElasticityIntegrator` has no partial or element assembly, and `DGDiffusionIntegrator` partial assembly works only on hexes.

4.4 Fault ranks and partition balance

The benchmark's `--partition-only` mode partitions the 200 m meshes with METIS, the same way as `ParMesh`. It then predicts each rank's step time from the measured per-tet cost and the marginal per-fault-face cost (traction twice plus slip load twice):

- p = 1: 3.4 µs per tet and 41.5 µs per fault face.
- p = 2: 15.7 µs per tet and 77.6 µs per fault face.

Mesh, ranks	Ranks with fault faces	Max fault-face sides on one rank	Predicted max/mean step time	Observed on Vista (rate vs local serial)
TPV205 200 m p1, 576	28%	650	1.90	1.1
TPV205 200 m p2, 2304	20%	231	1.57	1.57
TPV104 200 m p1, 512	34%	820	1.33	1.0
TPV104 200 m p2, 2048	24%	310	1.52	1.17

For the bulk alone, max/mean is 0.88 to 0.96, so METIS balances the tets well. The prediction is an upper bound: at p = 1 the Vista rates show little imbalance, so the real per-face cost there is lower than measured on the coarse local mesh. At p = 2 on 2048 to 2304 ranks, the observed excess (1.17 to 1.57×) is in the range the fault imbalance would cause (inferred; per-rank timers in the driver would confirm it). Item 3 shrinks the fault cost, and item 6 spreads what is left.

4.5 Fault output

The benchmark's `[H]` mode times `ParaViewHDFDataCollection::Save` with 12 fields. It was measured while gmsh ran on the same machine, so the absolute values are high.

Fault triangles	Deflate level 1	Deflate level 0	No filter
4.4k	185 ms	103 ms	27 ms
105k	1107 ms	706 ms	607 ms

perf on the 105k case puts about 60% in the per-cell field evaluation (`GetVectorValues`, `CalcPhysShape`) and about 25% in deflate and shuffle. The fault fields are already one value per cell, so writing the arrays directly removes the evaluation. Dropping deflate removes the compression cost, which matters more at the driver's default level 6. Writing the gathered vertices once, and leaving out the two constant fields, cuts the rest. Together these should cut the 0.48 to 1.59 s per frame several times over (inferred from the split above).

Asynchronous output, as SeisSol does with its ASYNC library (Rettenberger & Bader 2015, about 8× on wave-field output), would hide what remains.

4.6 Slivers set the time step

The time step is $\Delta t = \mathrm{cfl}\, C(p)\, \min_e h_e/(p^2 c_p)$, where $h_e = 6V_e/A_e$ (nitsche_cfl.hpp:69). The single worst tet therefore sets it for the whole mesh.

- On the TPV205 200 m mesh, $h_{\min}$ = 26.2 m. The median is 182 m; 0.1% of tets are below 43 m and 1% below 62 m. The Vista p = 2 time step implies $h \approx 30.6$ m.
- The smallest tets are slivers: on the 1000 m mesh the smallest has edges near 1000 m but h = 156 m.
- Adding `Mesh.Algorithm3D = 10; Mesh.OptimizeThreshold = 0.6; Mesh.OptimizeNetgen = 1;` to the 1000 m TPV205 geometry raises $h_{\min}$ from 156 to 262.5 m, a 1.68× larger dt. It also gives fewer tets (154k instead of 187k), so the work is about halved. `OptimizeThreshold = 0.6` alone gives 188 m.
- Track 2 now uses mirror meshes (mesh the half, reflect). The options must therefore go into the half-mesh step, and the 200 m meshes still need to be generated and checked. The benchmarks must be re-run on the new meshes.

Clustered local time stepping (rate 2) on the same 200 m mesh would give 1.99× with two clusters and 4.58× ideal with seven, since the smallest cluster holds 0.4% of tets. SeisSol gets 4.5 to 6.8× from LTS (Breuer et al. 2016; Uphoff et al. 2017). For SIPG with velocity damping, the schemes in Diaz & Grote (2009) and Grote & Mitkova (2013) apply. With a better mesh much of the LTS gain is already taken, so LTS is a later step.

4.7 Memory and setup

After both operators are assembled, resident memory is 4.4× the size of one CSR matrix. At p = 2 the setup peak is about 300 kB per tet: a 62.5k-tet p = 2 run used more than 13.9 GB locally and was killed. The extra memory is the kept serial forms (`cached_a_`), their local matrices, and the two unused AMG hierarchies. Freeing them after `ParallelAssemble` and building the operators without a solver would roughly halve memory (inferred from the 4.4× ratio). The p = 2 runs could then use fewer nodes, and setup would lose the AMG build. The two lazy assemblies take 13 to 16 s each on 62k p = 1 or 11k p = 2 tets.

5. Speed-up options in detail

5.1 Fault output (item 1)

- Write the L2 p = 0 cell arrays straight to the VTKHDF datasets, without `SaveGridFunction`.
- Write the geometry once and only append fields per frame.
- Drop `param_a` and `param_Dc` from the per-frame fields, or write them once.
- Make deflate off the default for fault output (the deck key `paraview_fault_deflate_level` already exists).
- Later: overlap the write with compute, using a non-blocking gather and a writer thread or dedicated rank (SeisSol ASYNC).

Supporting test: `[H]` in §6.

5.2 Mesh quality (item 2)

Add the gmsh optimiser options to the half-mesh scripts. Report $h_{\min}$, its percentiles and the tet count, then re-run the benchmarks. This is the largest gain that does not touch the solver, but it changes the mesh, so all golds must be redone.

5.3 Fault traction (item 3)

For each fault face, precompute at setup the small dense matrix that maps the two sides' DOFs to the projected traction DOFs. The per-step call is then one small matrix-vector product per face. This is how Tandem evaluates fault traction (Uphoff et al. 2023, GJI). The change would also:

- skip the unused decomposed diagnostics in the step,
- exchange only the DOFs of shared fault faces instead of the whole face-neighbour halo,
- reuse one buffer instead of allocating and copying a whole vector twice per step.

The benchmark builds this operator by probing the production call and matches it to round-off, so the result is unchanged.

5.4 Fused SpMV (item 4)

K and K_η have the same sparsity (the test checks this). One sweep over both value arrays, reading the column indices once, gives 1.10 to 1.26×. Hypre has no such kernel, so this needs a small custom CSR loop on the local diagonal and off-diagonal blocks, with the halo exchange of u and v done together. If the damping coefficient β were uniform, $K_\eta = \beta K$ and one SpMV of $K(u + \beta v)$ would do. Today β varies by a factor of about 100 across elements (scaled by element size, driver log), so this would change the damping model and is your decision, not part of this plan.

5.5 Memory and setup (item 5)

Build K and K_η with no solver. Release `cached_a_` and its local matrix once the `HypreParMatrix` exists, and assemble at construction instead of on the first step.

5.6 Partition weights (item 6)

Pass element weights to the partitioner: base 1 per tet and about 12 (p = 1) or 5 (p = 2) extra per fault face. These follow from the measured per-face to per-tet cost ratios (41.5/3.4 and 77.6/15.7). SeisSol uses 100 + 200 per rupture face (Krenz et al. 2021, SC21). The benchmark's `--partition-only` mode can predict the result before any run.

5.7 Larger projects (items 7 and 8)

- **Clustered LTS** (§4.6). Use a leapfrog LTS for second-order systems with damping (Diaz & Grote 2009; Grote, Michel & Sauter 2021 for the stabilised form). Elements next to a fault face must share a cluster, as in SeisSol.
- **Custom SIPG kernels.** Store per-element and per-face dense blocks, or go fully matrix-free with reference matrices and per-element geometry, as Tandem and SeisSol do with YATeTo (Uphoff & Bader 2020). Dense element blocks measured 1.13 to 1.44× on the

volume part. A matrix-free kernel would cut memory traffic about 10× at the cost of more flops (estimate from the research notes). MFEM has nothing to reuse here on tets.

- **Other options:** single precision for $K_\eta v$, OpenMP threading or GPUs. None of these is backed by a test here.

6. Supporting test

`miniapps/seas/tests/unit/bench_nitsche_kernels.cpp` (new, uncommitted, with a `seas_bench_nitsche_kernels` Makefile target) builds the production `ElasticityDomainOperator` for K and K_η, using the driver's β formula, on any mesh. It has these sections:

- `[S]`: memory bandwidth (stream triad).
- `[K]`: production SpMV time, nnz, bandwidth and resident memory.
- `[F]`: the fused sweep, with a sparsity-equality check.
- `[V]`: volume-part variants (full, MFEM partial assembly, dense blocks).
- `[T]`: traction (current, without diagnostics, precomputed per face) and slip load.
- `[P]`: METIS partition balance.
- `[H]`: fault VTKHDF frame cost.

All nine equivalence checks pass (the rebuilt K and K_η equal the production ones, the fused, partial-assembly and dense-block products match, and the precomputed traction matches). The program exits non-zero if any check fails.

```
cd miniapps/seas
make MFEM_DIR=../.. HDF5_DIR=$HDF5 seas_bench_nitsche_kernels
./seas_bench_nitsche_kernels --mesh MESH.msh --order 1 --eta 0.1 --reps 20
./seas_bench_nitsche_kernels --mesh tpv205/mesh/tpv2053d_200m.msh --partition-only \
    --partitions 576,2304 --c-tet 3.4e-6 --c-face 4.15e-5
./seas_bench_nitsche_kernels --mesh ... --hdf-dir OUT --hdf-cells 105000 --hdf-fields 12 --
hdf-frames 5
```

7. Where it lives

- Code read at `53ba400` (nitsche-benchmarks). Local branch `nitsche-profiling` holds the benchmark and its Makefile target. Nothing is committed or pushed yet.
- Vista logs: `results/driver.log` in the gold folders for jobs 1076378, 1076419, 1075919, 1075916 and 1075917.
- This report: `/mnt/project-files/profiling/nitsche-profiling-plan-rev1.pdf` and its source HTML.
- Literature: Breuer, Heinecke & Bader (2016) IPDPS; Uphoff et al. (2017) SC17; Krenz et al. (2021) SC21; Rettenberger & Bader (2015) CLUSTER; Uphoff & Bader (2020) ACM TOMS; Uphoff, May & Gabriel (2023) GJI 233; Diaz & Grote (2009) SISC 31; Grote & Mitkova (2013) JCAM 239; Grote, Michel & Sauter (2021) Math. Comp.; Pelties et al. (2012) JGR 117; Anderson et al. (2021) CAMWA 81; Kronbichler & Kormann (2019) ACM TOMS 45; Moxey, Amici & Kirby (2020) SISC 42.